Học viện Công nghệ Bưu chính Viễn thông
Khoa Công nghệ Thông tin 1

Nhập môn học sâu

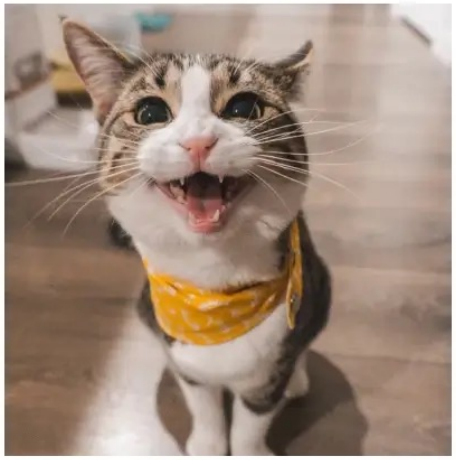
Mạng nơ ron tích chập

TS. NGUYỄN XUÂN ĐỨC

Nội dung

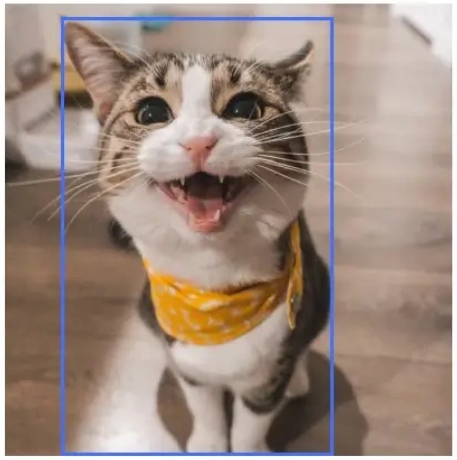
- Mở đầu
- Phép tương quan chéo
- Kiến trúc mạng CNN
- Huấn luyện mạng CNN
- Ứng dụng CNN
- Mạng CNN hiện đại

Mở đầu



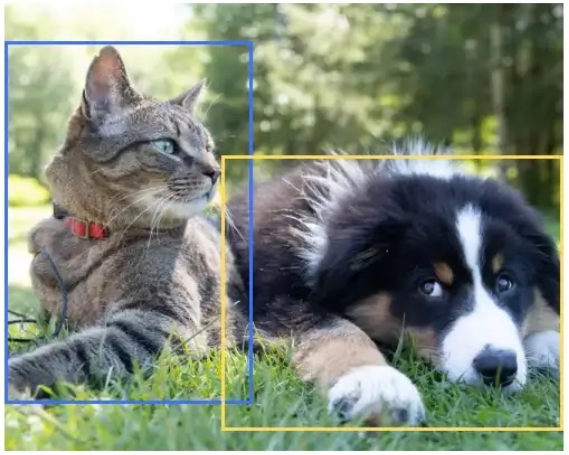
Classification

Cat



Classification, Localization

Cat



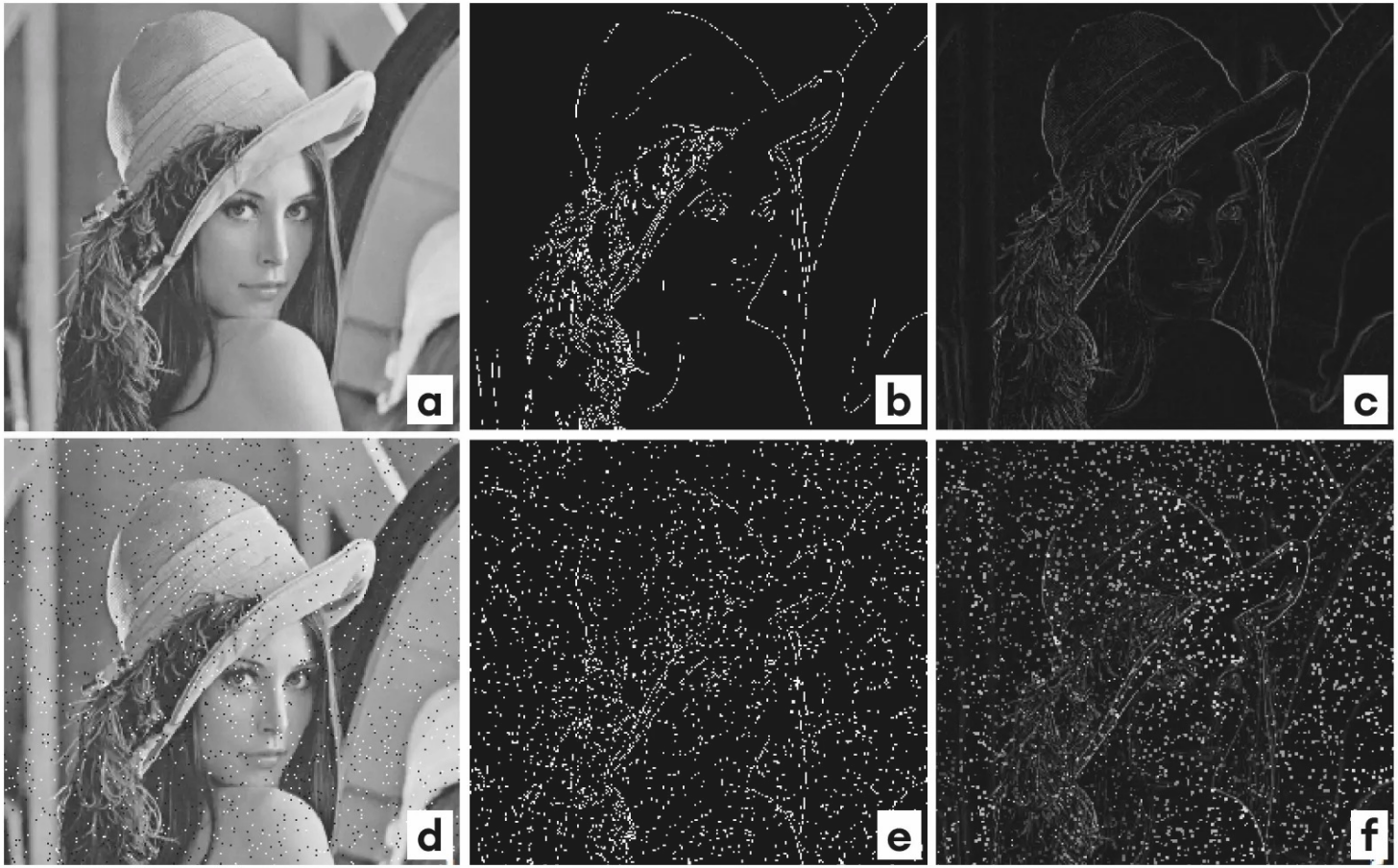
Object Detection

Cat, Dog

Mở đầu

- Giả sử ảnh đầu vào có kích thước 1000x1000
- Nếu dùng mạng MLP với 1000 nút ẩn, số lượng tham số đạt tới 10^9!
- MLP: mạng nơ ron không có cấu trúc
- Giải pháp: Mạng nơ ron tích chập (CNN)
  - Chia sẻ tham số: giảm số lượng tham số vì mỗi kernel được sử dụng trên toàn bộ bức ảnh
  - Có thể áp dụng phép tích chập với kích thước ảnh bất kỳ

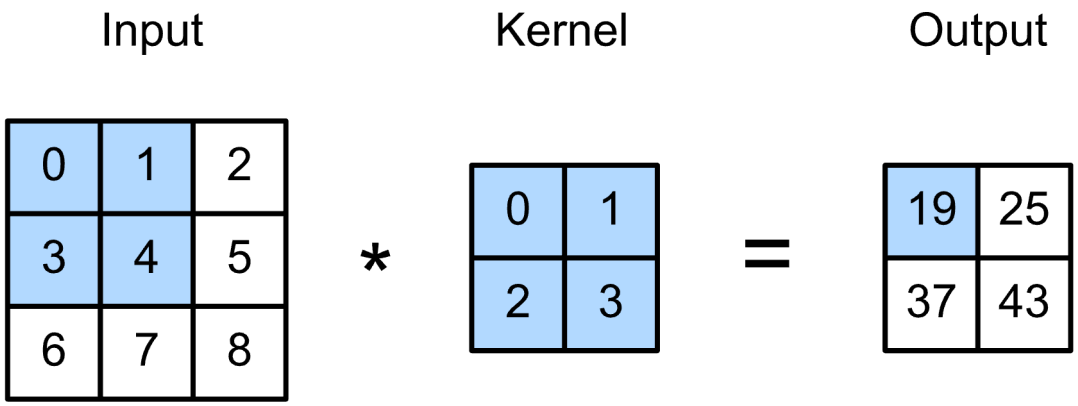
Nhận diện cạnh



Phép tương quan chéo

- Một tensor đầu vào và bộ lọc (kernel) được kết hợp để tạo ra tensor đầu ra
- Cửa sổ tích chập được trượt từ góc trái trên, trái sang phải và trên xuống dưới
- Kích thước đầu ra được tính theo công thức:

$$(H - h + 1) \times (W - w + 1)$$



Lớp tích chập

- Lớp tích chập áp dụng phép tương quan chéo giữa đầu vào và bộ lọc, cộng thêm bias

```
class Conv2D(nn.Block):
    def __init__(self, kernel_size, **kwargs):
        super(Conv2D, self).__init__(**kwargs)
        self.weight = self.params.get('weight', shape=kernel_size)
        self.bias = self.params.get('bias', shape=(1,))

    def forward(self, x):
        return corr2d(x, self.weight.data()) + self.bias.data()
```

Học tham số của bộ lọc

PYTORCH MXNET JAX TENSORFLOW

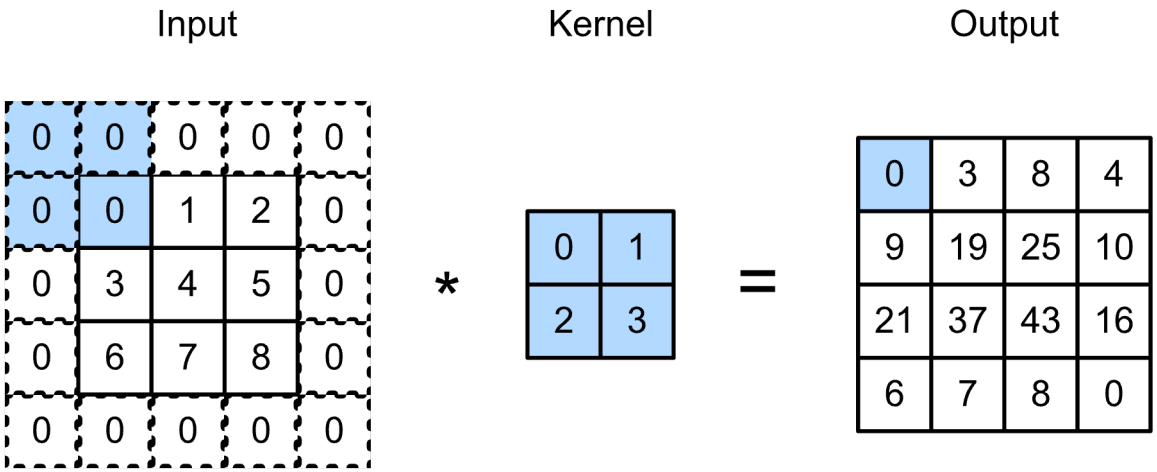
```python
# Construct a two-dimensional convolutional layer with 1 output channel and a
# kernel of shape (1, 2). For the sake of simplicity, we ignore the bias here
conv2d = nn.LazyConv2d(1, kernel_size=(1, 2), bias=False)

# The two-dimensional convolutional layer uses four-dimensional input and
# output in the format of (example, channel, height, width), where the batch
# size (number of examples in the batch) and the number of channels are both 1
X = X.reshape((1, 1, 6, 8))
Y = Y.reshape((1, 1, 6, 7))
lr = 3e-2  # Learning rate

for i in range(10):
    Y_hat = conv2d(X)
    l = (Y_hat - Y) ** 2
    conv2d.zero_grad()
    l.sum().backward()
    # Update the kernel
    conv2d.weight.data[:] -= lr * conv2d.weight.grad
    if (i + 1) % 2 == 0:
        print(f'epoch {i + 1}, loss {l.sum():.3f}')
```

```
epoch 2, loss 16.481
epoch 4, loss 5.069
epoch 6, loss 1.794
epoch 8, loss 0.688
epoch 10, loss 0.274
```

Padding

- Khi áp dụng lớp tích chập, chúng ta có thể mất một số điểm ảnh trên biên của ảnh. Sự mất mát này được cộng dồn khi có nhiều lớp tích chập.
- Giải pháp: chèn thêm điểm ảnh xung quanh biên của ảnh
- Kích thước đầu ra sau khi thêm p_h hàng và p_w cột:

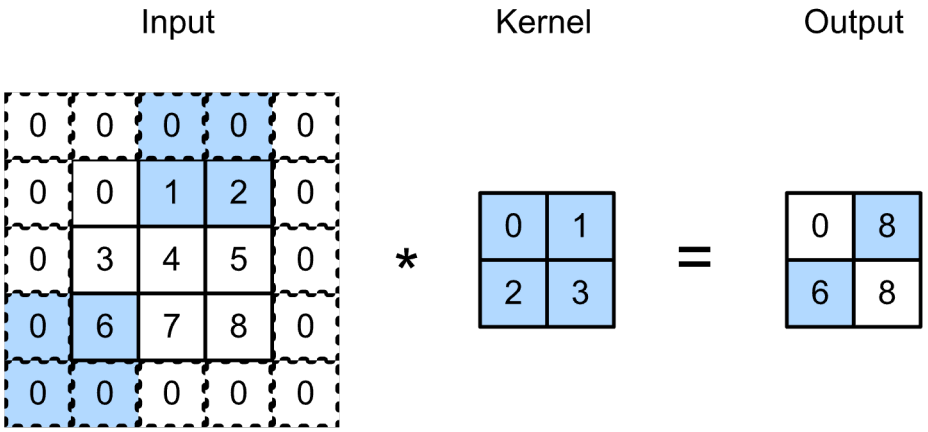
$$(n_h - k_h + p_h + 1)\times(n_w - k_w + p_w + 1)$$



Stride

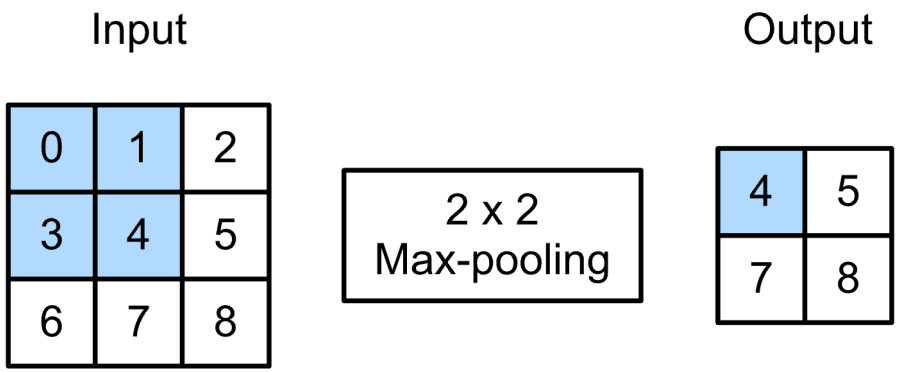
- Stride: số hàng và cột mà bộ lọc di chuyển tại mỗi bước
- Kích thước đầu ra:

$$\left\lfloor \frac{n_h - k_h + p_h + s_h}{s_h} \right\rfloor \times \left\lfloor \frac{n_w - k_w + p_w + s_w}{s_w} \right\rfloor$$



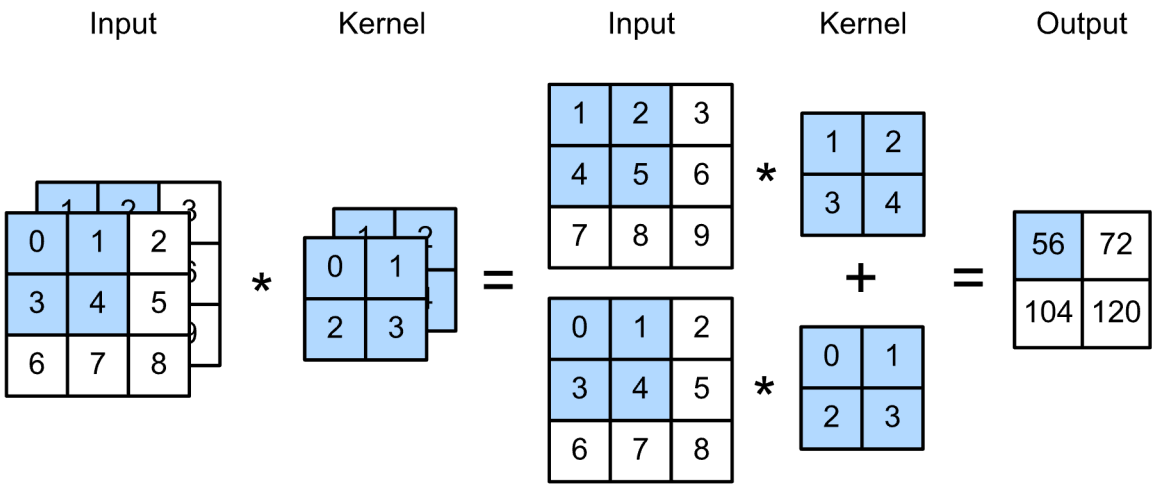
Pooling

- Phép *pooling* tính một giá trị đầu ra duy nhất tại mỗi vị trí cửa sổ trượt qua
- Các phép pooling: max pooling, average pooling



Đa kênh đầu vào

- Ảnh đầu vào có thể có nhiều kênh: ảnh RGB có kích thước $3 \times h \times w$
- Gọi số kênh đầu vào là c_i, ta cần c_i bộ lọc tích chập
- Ví dụ: 2 kênh đầu vào, 1 kênh đầu ra

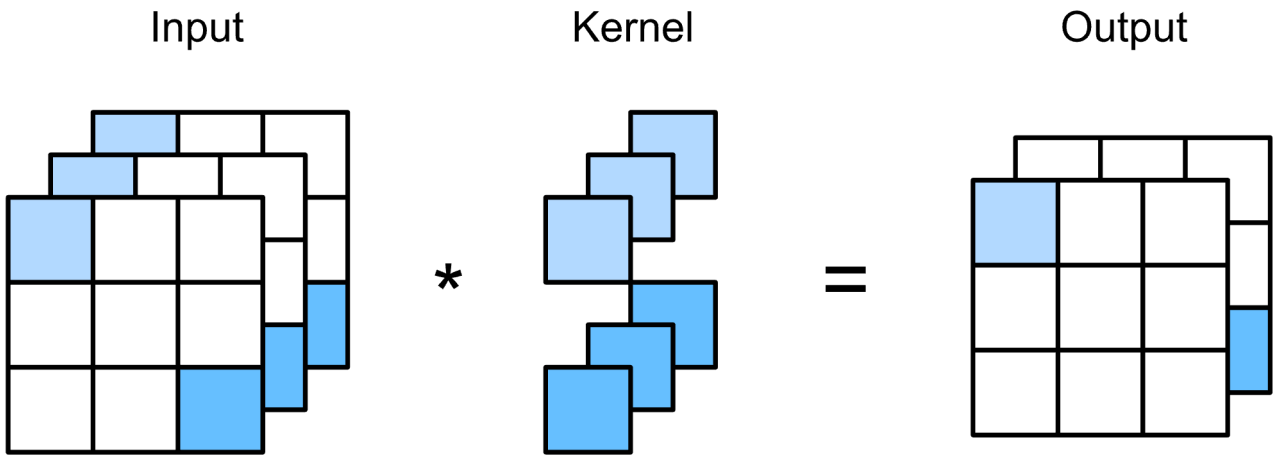


Đa kênh đầu ra

- Thông thường, ở các tầng sâu hơn:
  - Tăng số kênh đầu ra
  - Giảm kích thước ảnh
- Gọi c_i và c_o là số lượng kênh đầu vào và đầu ra, k_h và k_w là chiều cao và chiều rộng của bộ lọc
  - Số lượng tham số: c_i x c_o x k_h x k_w

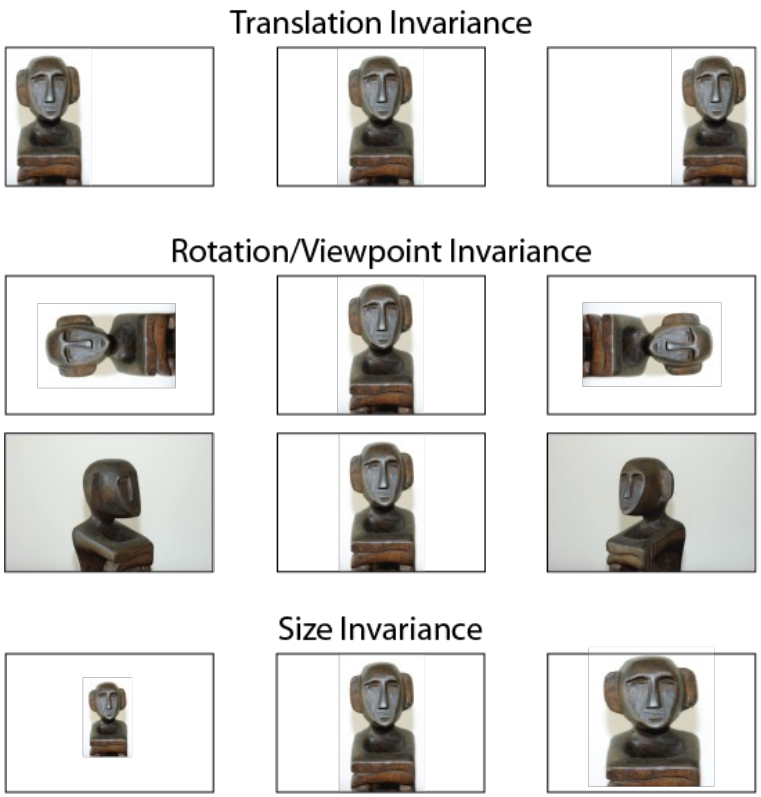
Lớp tích chập 1x1

- Phổ biến trong mạng CNN sâu
  - Không học tương quan giữa các điểm ảnh
  - Ứng dụng: giảm số kênh đầu ra



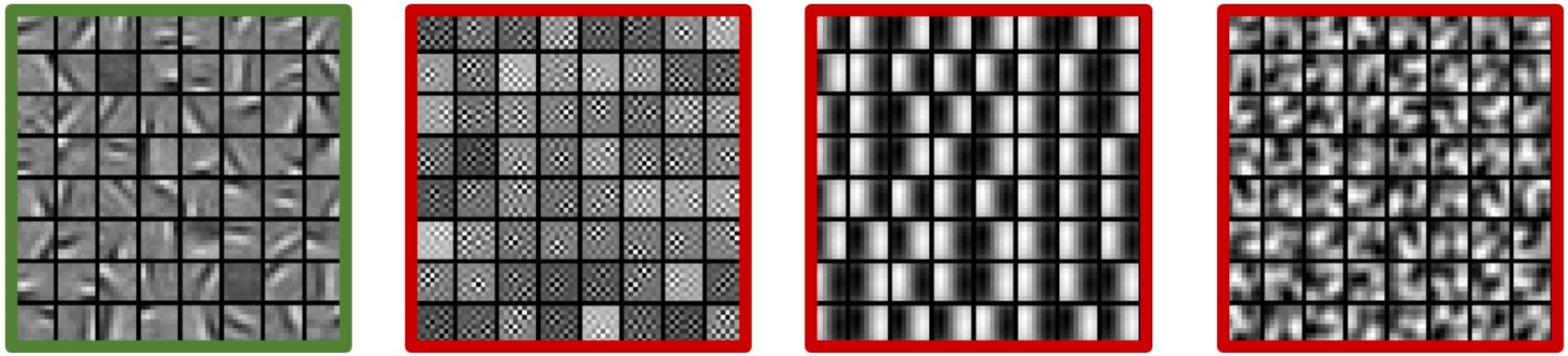
Tính bất biến của CNN

- Khi vật thể có thay đổi, mạng CNN vẫn có thể nhận diện:
  - Dịch chuyển (Translation)
  - Xoay (Rotation)
  - Kích thước (Size)



Kiểm tra trọng số

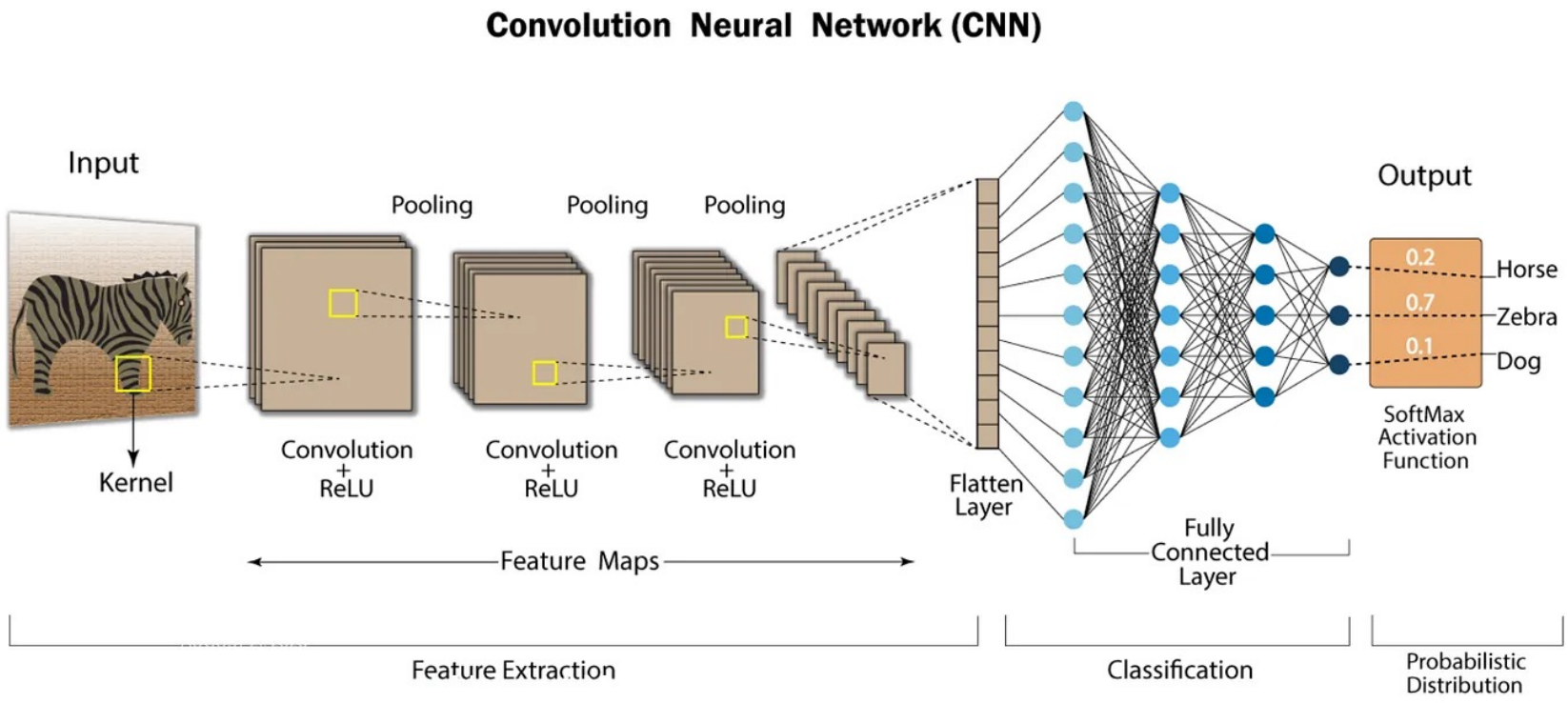
- Trọng số của bộ lọc có thể được dùng để xác định xem CNN có học được thông tin hữu ích hay không
- Tham khảo:
  - https://distill.pub/2017/feature-visualization/



GOOD | BAD (noisy) | BAD (correlated) | BAD (no structure)

Huấn luyện CNNs

Phân loại ảnh dùng CNNs



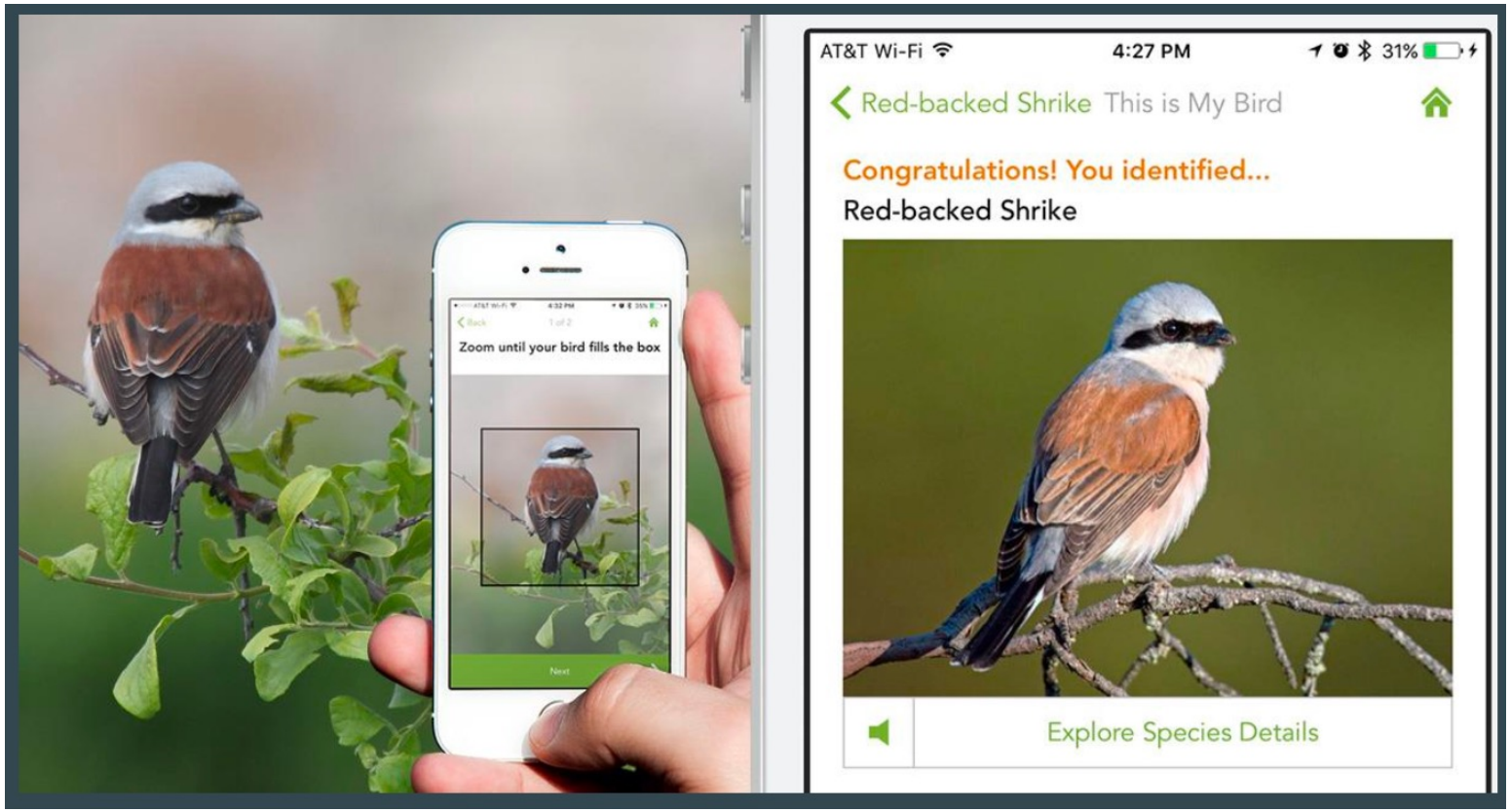
Kiến trúc cơ bản để phân loại ảnh

Siêu tham số

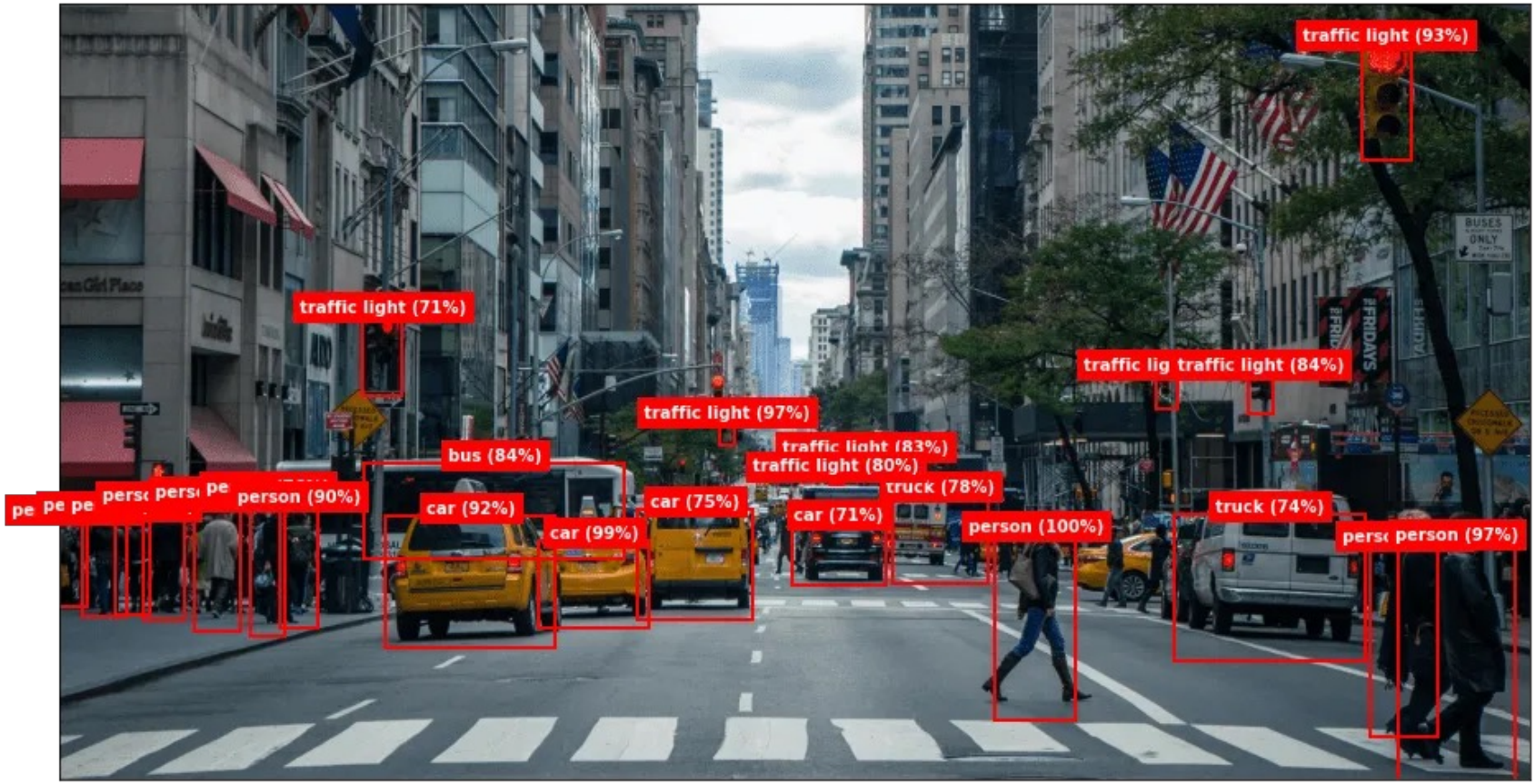
- Lớp tích chập: C_i x C_o x H x W
- Hàm kích hoạt: ReLU, sigmoid, tanh
- Hàm mất mát: cross entropy (phân lớp), MSE (hồi quy)
- Điều chuẩn: Dropout, BatchNorm

Ứng dụng CNN

Phân loại ảnh

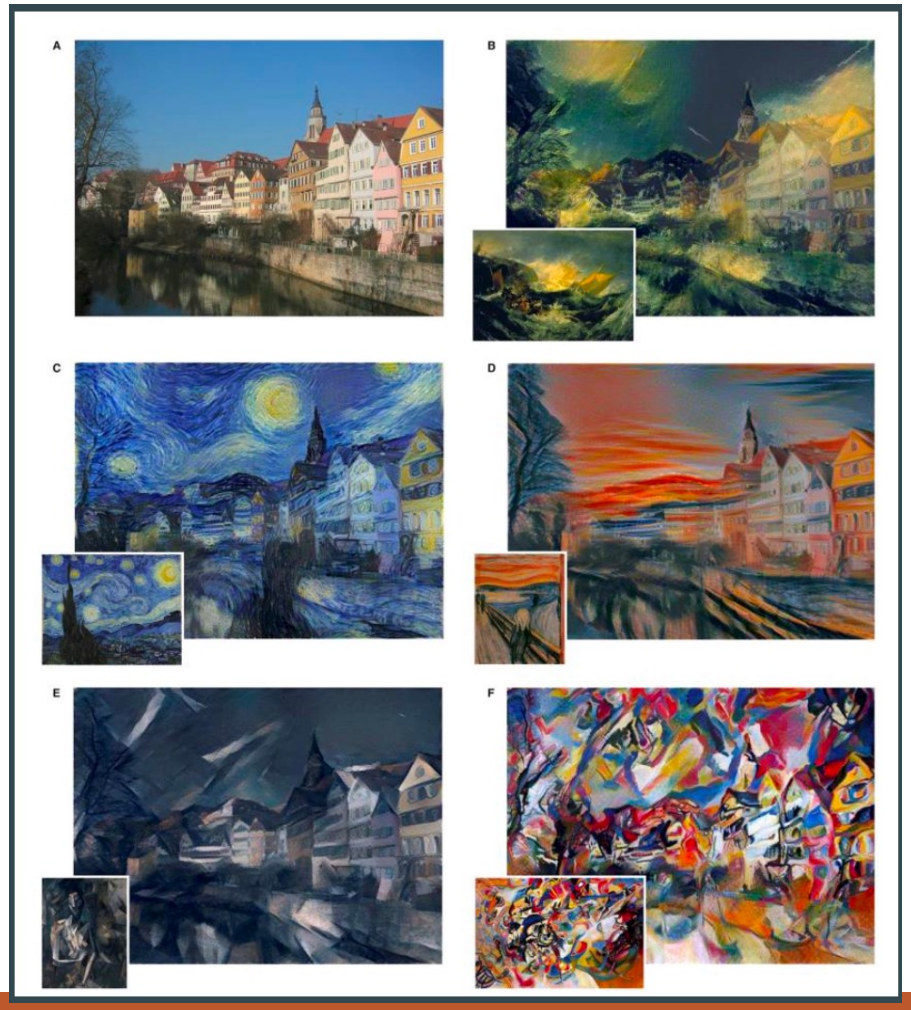


Nhận diện vật thể



Phân đoạn ảnh

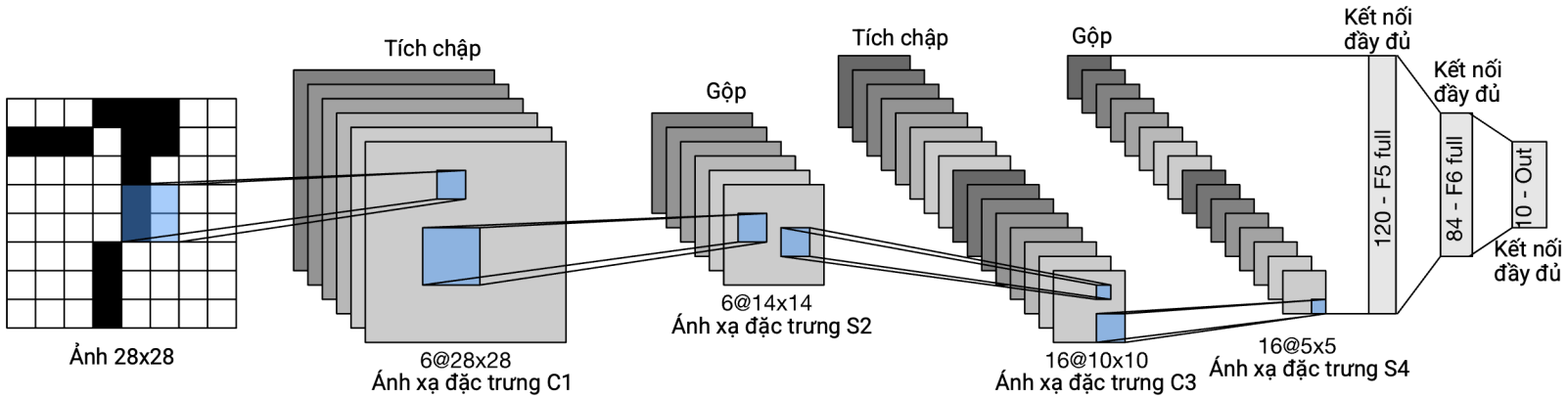
Style transfer



Mạng CNN hiện đại

LeNet (1989)

- Conv: 5x5, pooling: 2x2, tanh activation
- <1% tỉ lệ lỗi trên tập MNIST
- Đầu ra là vector 10 chiều
- 60K tham số

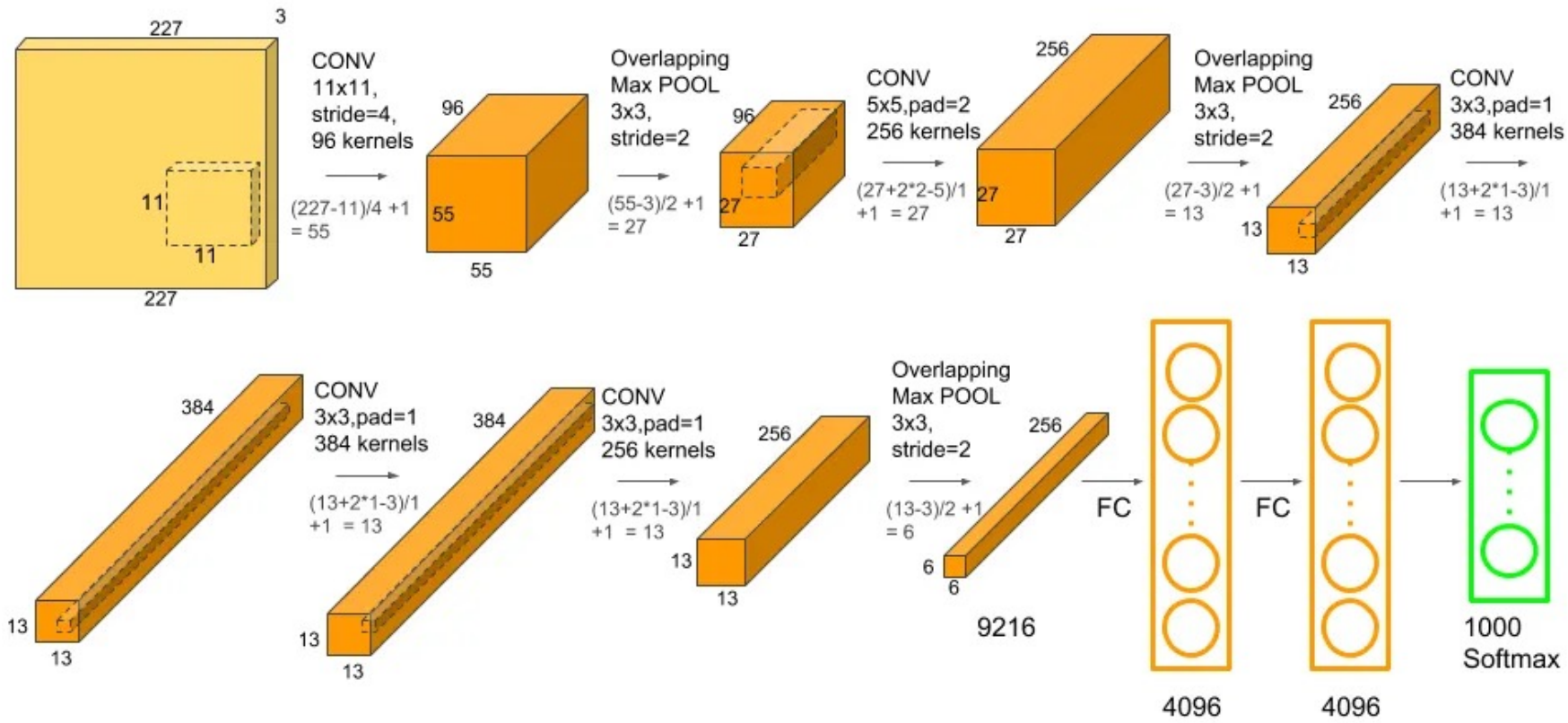


LeNet demo (1989)

AlexNet (2012)

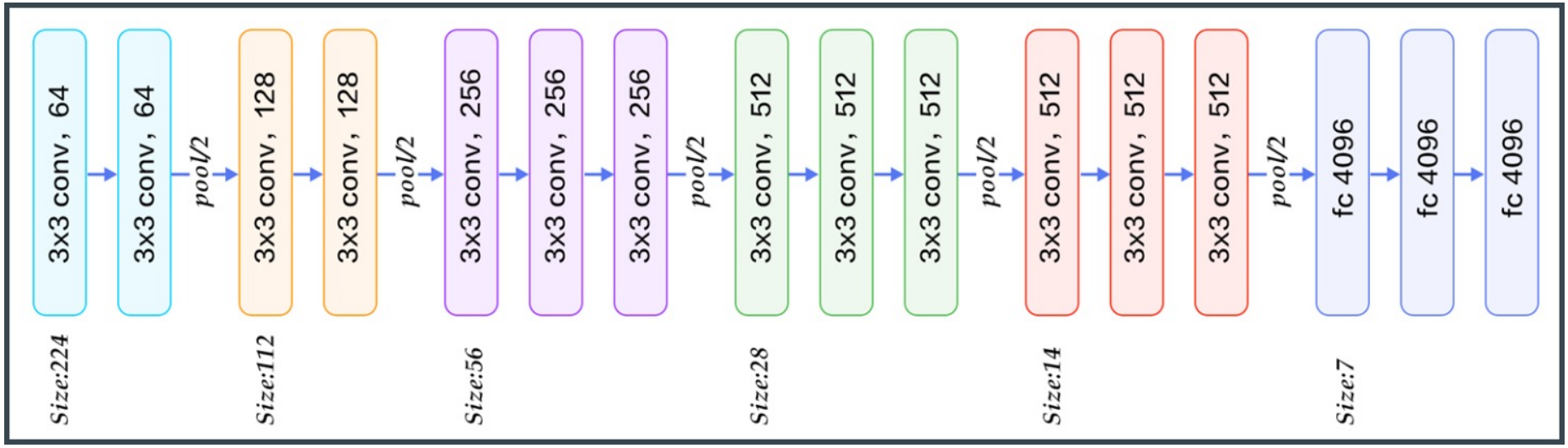
- Conv: 11x11, 5x5, 3x3, pooling: 3x3, ReLU activation
- Điều chuẩn: Dropout
- Cải thiện trên tập ImageNet: 15.3% tỷ lệ lỗi so với 26.2% (second best)
- Cài đặt trên GPU
- 60M tham số

AlexNet (2012)



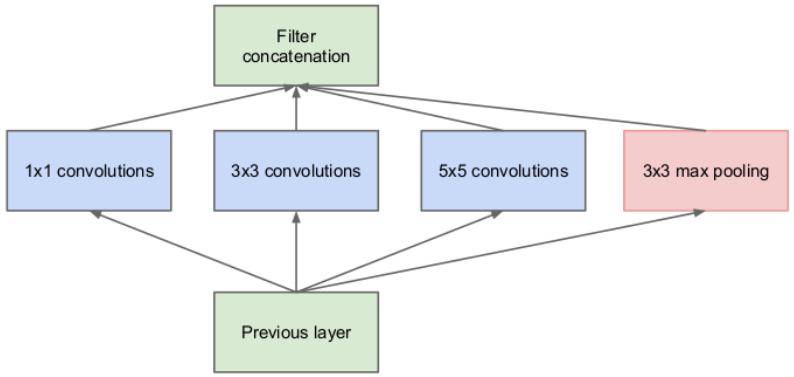
VGGNet (2014)

- Conv: 3x3, pooling: 2x2, ReLU activation
- 138M tham số

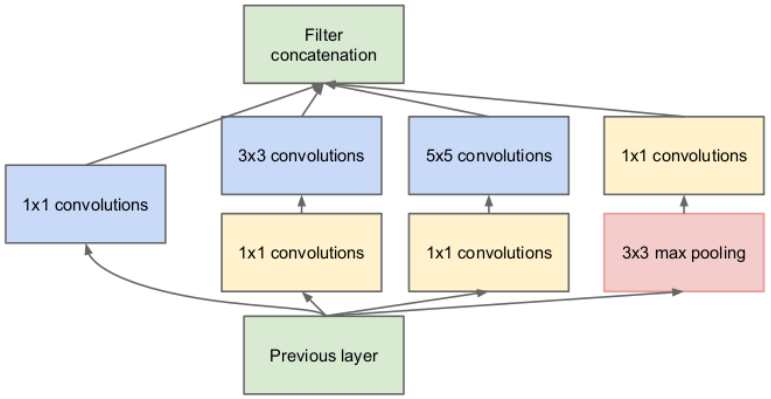


GoogLeNet (Inception) (2014)

- Ý tưởng: dùng nhiều bộ lọc với kích thước khác nhau cho cùng đầu vào



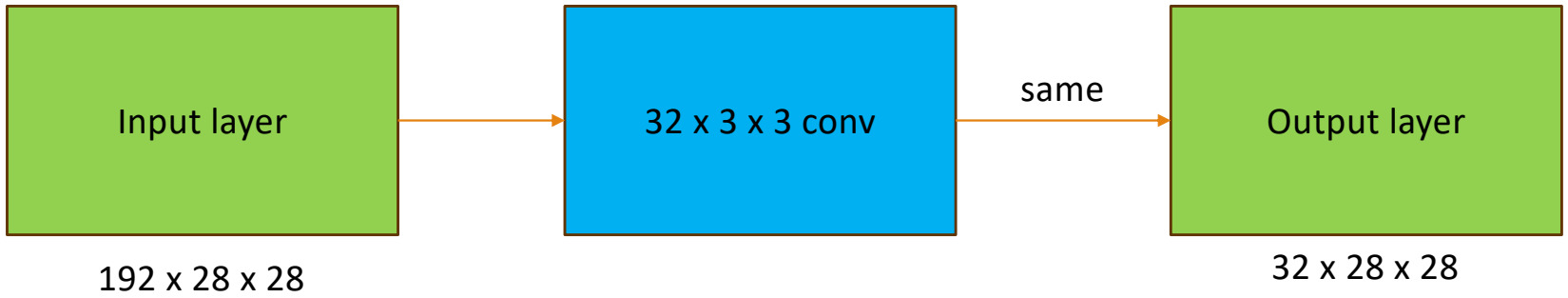
(a) Inception module, naïve version



(b) Inception module with dimension reductions

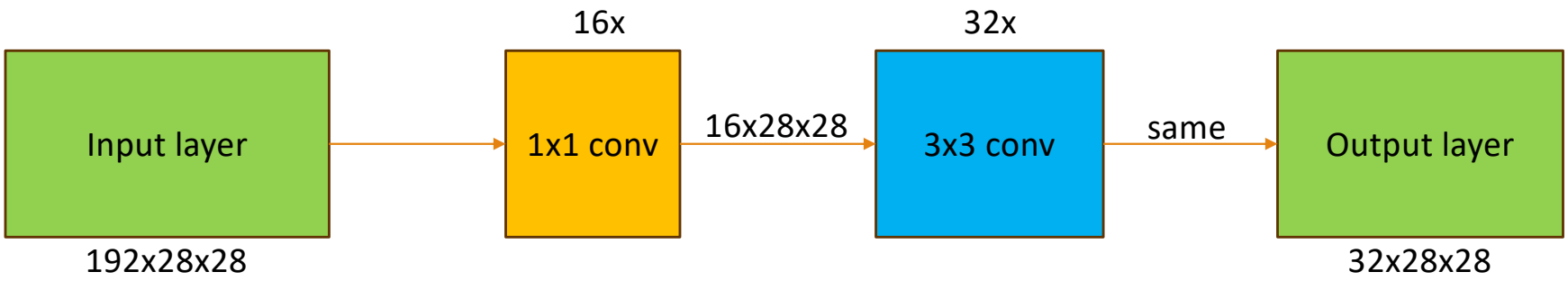
Inception (naive)

- Số lượng tham số: 192 x 32 x 3 x 3 ~ 55K



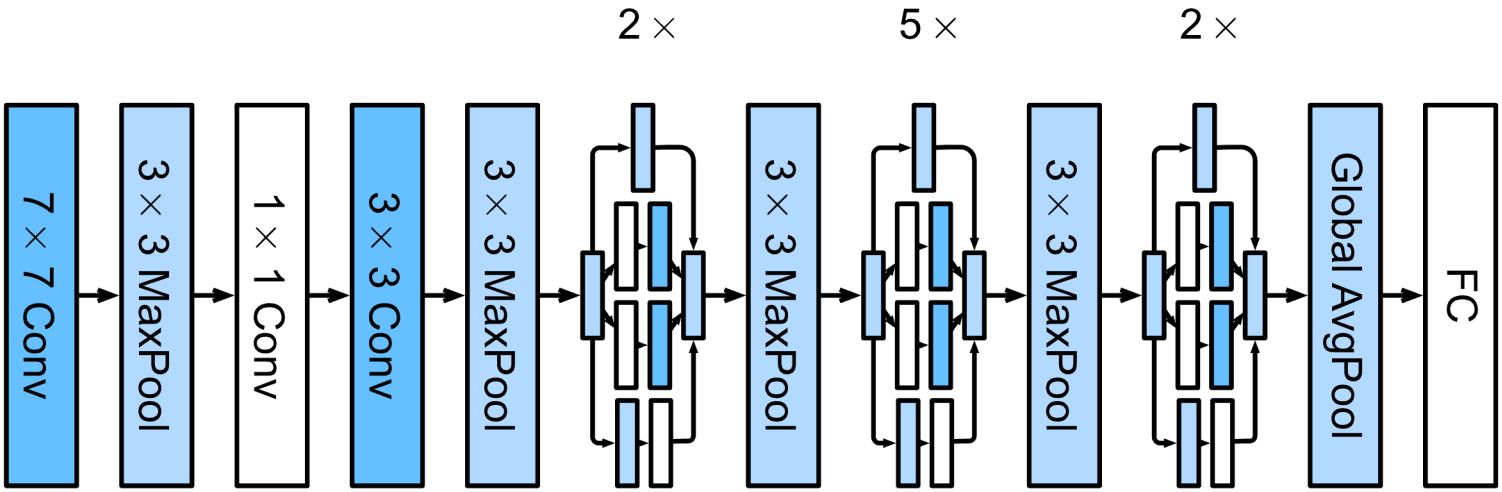
Inception (dimension reduction)

- Số lượng tham số:
  - Tầng 1: 192 x 16 x 1 x 1 ~ 3K
  - Tầng 2: 16 x 32 x 3 x 3 ~ 4.6K
  - Tổng: 7.6K



GoogLeNet

- GoogLeNet dùng 9 khối Inception:



BatchNorm

- BatchNorm (BN) chuyển đổi giá trị kích hoạt tại tầng x theo công thức

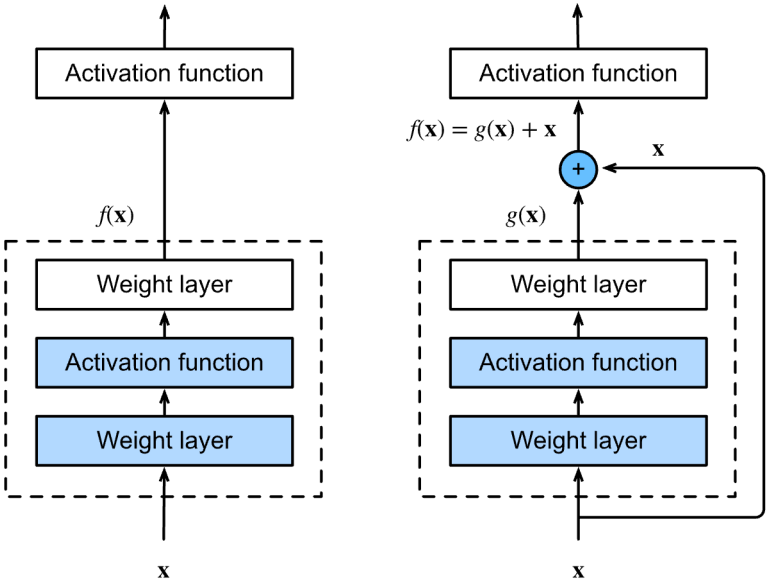
$$BN(x) = \gamma \odot \frac{x - \hat{\mu}}{\hat{\sigma}} + \beta$$

trong đó $\hat{\mu}$ là giá trị trung bình và $\hat{\sigma}$ là độ lệch chuẩn của các mẫu trong minibatch, γ, β là tham số của BN

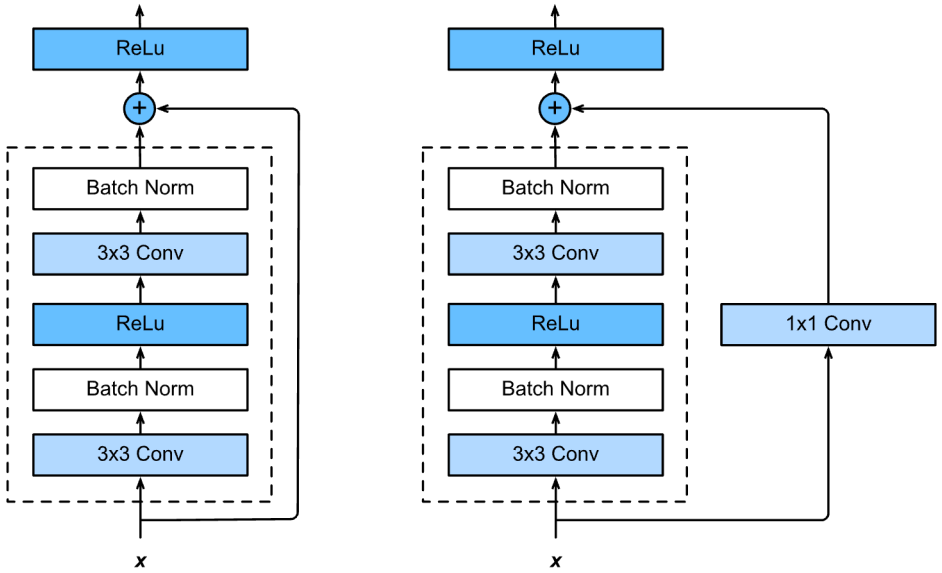
- Thứ tự sử dụng BN: Conv/FC → BN → Activation → Dropout

ResNet (2015)

- Khối phần dư: để tăng số lớp của mạng, đầu vào x được truyền trực tiếp đến đầu ra



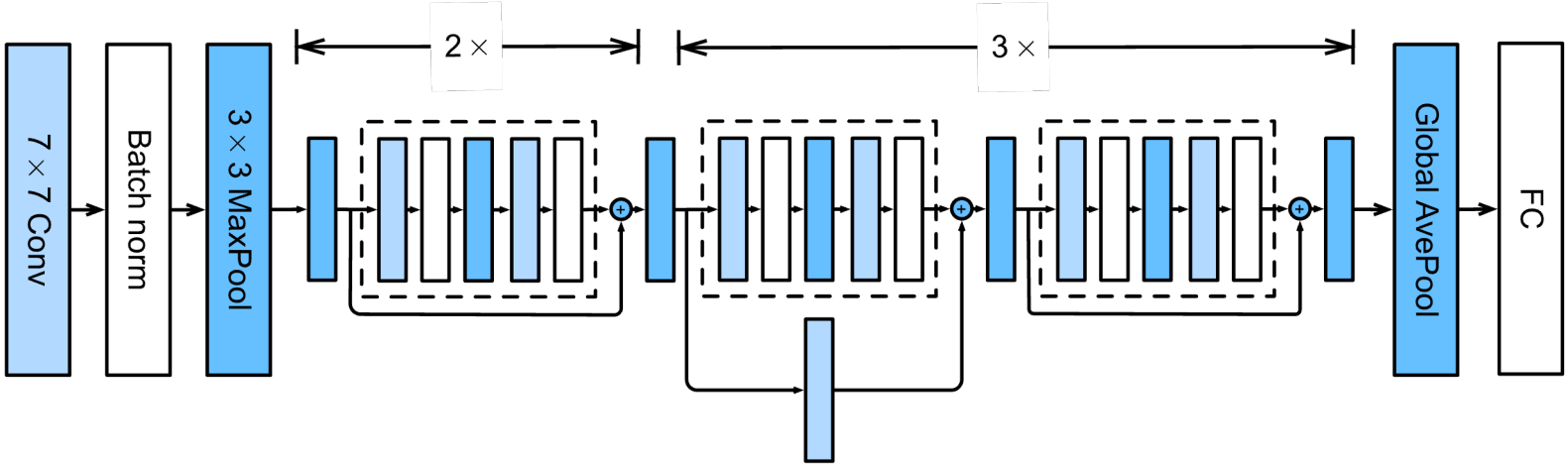
Khối phần dư



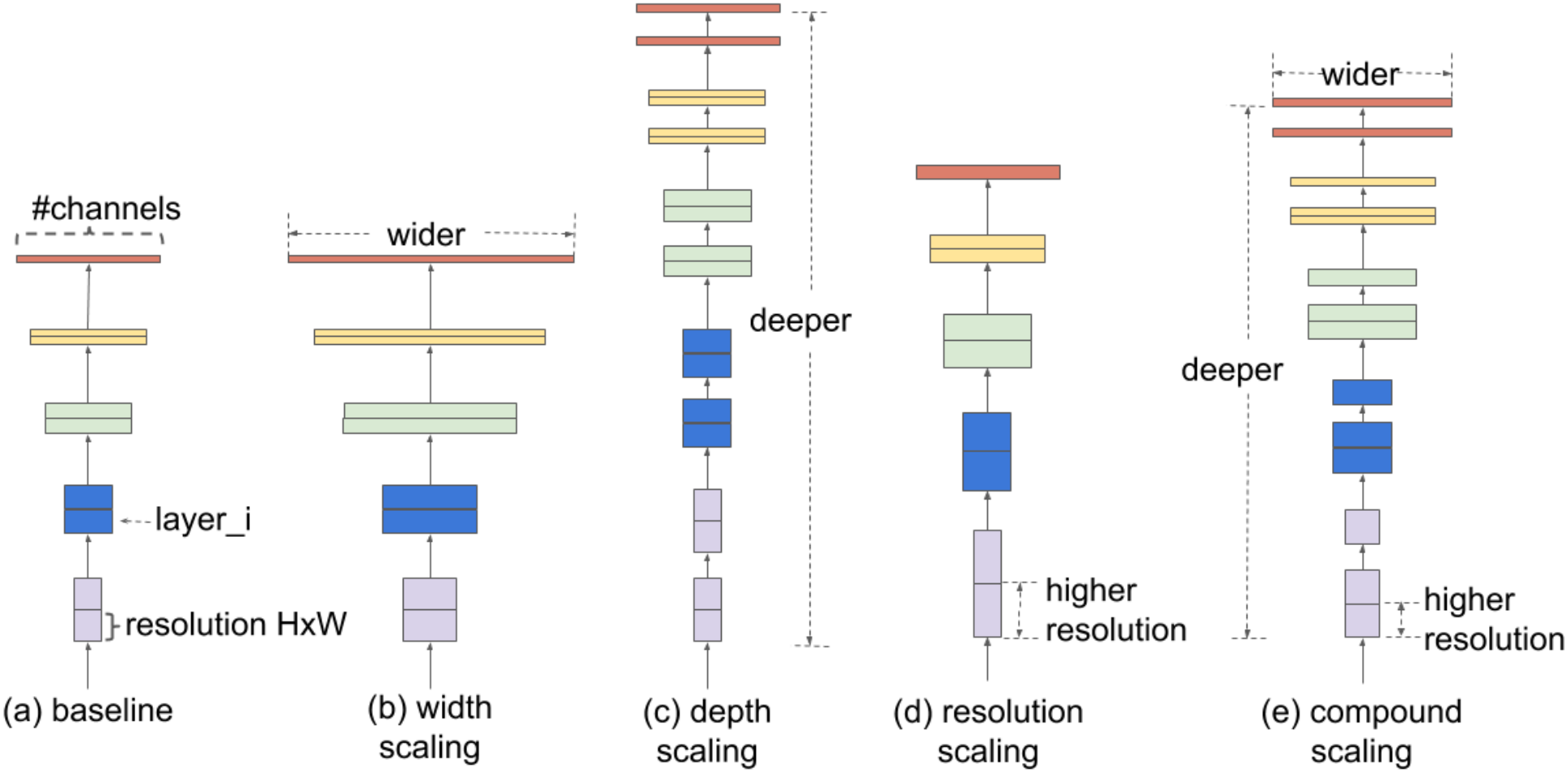
ResNet block

ResNet

- Mạng ResNet kết hợp các khối ResNet như sau:



EfficientNet



Luyện tập

- Phân loại ảnh sử dụng MNIST